

CO4- Assessment - 01

Comparative Analysis

CSAO307 - Data Structures

Name : M. Devadharshini

Register no: 192572149.

1. Binary Heap Vs Fibonacci Heap

For a graph with 100000 nodes and 10 million edges:

Binary Heap:

decrease-key() - $O(\log N)$

extract-min() - $O(\log N)$

Dijkstra complexity - $O(V^2 \log V)$

~~27 lines~~ less memory and is easy to implement.

Provides good practical performance due to better cache locality.

Fibonacci Heap:

decrease-key $\rightarrow O(1)$ amortized

extract-min $\rightarrow O(\log V)$ amortized.

Dijkstra complexity - $O(E + V \log V)$

It has better theoretical performance, especially for graphs with many edges.

However, it requires more memory and has a complex implementation.

Why Fibonacci heaps are rarely used?

Although Fibonacci heaps are better theoretical complexity, they are rarely preferred because of high memory overhead, complex pointer operations, poor cache locality & larger constant factors.

Binary heaps are simpler and often faster in real-world applications.

Conclusion:

Fibonacci heap is theoretically better, but binary heap is usually more practical.

2. Timsort vs Merge sort

Input	Timsort	Merge sort
Already sorted	$O(n)$	$O(n \log n)$
Reverse sorted	$O(n)$	$O(n \log n)$
Many duplicates	$O(n) - O(n \log n)$	$O(n \log n)$
Random order	$O(n \log n)$	$O(n \log n)$
Worst case	$O(n \log n)$	$O(n \log n)$

a) Already sorted

Timsort detects the existing sorted run and requires approximately $O(n)$ time. Merge sort still performs its normal divide and merge process: $O(n \log n)$.

b) Reverse sorted

Timsort detects descending run and reverses them, giving approximately $O(n)$ performance. Merge sort remains $O(n \log n)$.

c) Many Duplicate keys.

Timsort can benefit from long runs of equal or ordered values and may perform between $O(n)$ and $O(n \log n)$. Merge sort remains $O(n \log n)$.

/ Random Order:

Both algorithms generally require $O(n \log n)$ time.

Conclusion:

TimSort is generally better for real-world data because it takes advantage of already sorted or partially sorted data, while maintaining $O(n \log n)$ worst-case performance.

